Ejercicio_01
Tipos e interfaces en TypeScript con Angular

Modelado de datos de productos con interfaces y creación de un componente standalone

Objetivo

Practicar la definición de tipos con interfaces de TypeScript y la creación de un componente standalone de Angular que muestre los datos de un producto, organizando el proyecto siguiendo el patrón de carpetas core / shared / features.

En la primera parte de este ejercicio todavía NO se usan directivas de control de flujo (@for, @if,...). El objetivo es trabajar únicamente con tipos, propiedades de clase y binding de datos en la plantilla.

Contexto: los datos

Como referencia de datos se usará la API pública Fake Store (https://fakestoreapi.com/products). No es necesario hacer ninguna petición HTTP: los datos se trabajarán como una constante local en el proyecto.

Consulta en el navegador la URL https://fakestoreapi.com/products y observa la forma de cada producto: qué campos tiene, de qué tipo es cada uno, y qué estructura tiene el campo "rating".

Enunciado

1. Estructura de carpetas

Dentro de src/app, crea la siguiente organización de carpetas:

- Core
- shared
- features

2. Enumerado de categorías (Category) e interfaces de datos

Define las interfaces necesarias en coer/models para tipar los datos de productos

Antes de definir la interfaz Product, observa los valores que toma el campo category en los productos de la API: solo hay 4 categorías distintas. En vez de dejar ese campo como string libre, crea primero un enumerado en core/models que restrinja category a esos 4 valores concretos.

- Crea un enum de tipo string llamado Category, con un miembro por cada categoría real de la API (por ejemplo, el miembro debe tener asignado el texto exacto que devuelve la API, como "electronics").

Con el enum ya creado, define en core/models dos interfaces de TypeScript a partir de los datos observados en la API:

- Rating: representa la valoración de un producto (nota media y número de valoraciones).
- Product: representa un producto (identificador, título, precio, descripción, categoría, imagen y valoración). La propiedad de valoración debe usar el tipo Rating y la de categoría debe usar el tipo Category.

Piensa bien el tipo de cada propiedad (number, string...) según el valor real que devuelve la API.

3. Array de datos de ejemplo

En core/mocks, crea un archivo con una constante exportada llamada PRODUCTS, tipada como un array de Product (Product[]), que contenga los productos de la API (puedes copiar los datos reales que devuelve el endpoint), usando en cada producto el miembro del enum Category que corresponda.

El array debe estar completamente tipado: si añades un producto con un campo mal escrito o de tipo incorrecto, el compilador de TypeScript debe marcarlo como error.

4. Imágenes en local

Descarga las imágenes de los productos y guárdalas dentro de public/images, nombrando cada archivo con el id del producto y extensión .png (por ejemplo, public/images/1.png para el producto con id 1).

Actualiza el campo image de cada producto en tu array PRODUCTS para que apunte a la ruta local (por ejemplo, "/images/1.png") en lugar de a la URL de la API.

5. Componente Products

Dentro de features/products, crea un componente standalone llamado Products, con sus tres archivos separados (.ts, .html y .css). El componente debe cumplir:

- Seleccionar un único producto del array PRODUCTS accediendo por índice (por ejemplo, el tercer producto del array), y guardarlo en una propiedad de la clase tipada como Product.
- Usar ChangeDetectionStrategy.OnPush.
- En la plantilla, mostrar: la imagen del producto, un título con el texto "Producto" seguido del id del producto seleccionado, el título (title) del producto, la categoría, el precio y la valoración (nota y número de opiniones).
- Usar la directiva NgOptimizedImage para la imagen, ya que es un recurso estático servido desde public/.
- No usar directivas de control de flujo (@for, @if): al mostrar un solo producto no son necesarias.

6. Integración en la aplicación

Sustituye la plantilla de bienvenida que genera "ng new" en AppComponent por la llamada al componente Products, manteniendo la etiqueta <router-outlet /> para futuras rutas.

Criterios de evaluación

- El proyecto compila sin errores de tipos (ng build / tsc --noEmit).
- Las interfaces Product y Rating reflejan correctamente los datos de la API.
- El array PRODUCTS está correctamente tipado como Product[].

- Las imágenes se sirven en local desde public/images.
- El componente Products muestra correctamente los datos de un producto concreto, incluyendo su id en el título.
- El campo category usa el enumerado Category en la interfaz Product y en todos los productos del array PRODUCTS.
- No se usa ninguna directiva de control de flujo.
- Se respeta la estructura de carpetas core / shared / features.

Segunda parte: búsqueda de productos por id

En esta segunda parte, el componente Products deja de mostrar un producto fijo y permite buscar un producto del array PRODUCTS por su id. La búsqueda se hará de tres formas distintas, colocadas una al lado de la otra, para comparar cómo llega un valor de la vista al componente.

En esta parte sí se usa la directiva de control de flujo @if. NO se usan formularios (<form>, Reactive Forms) ni signals: el estado del componente se guarda en propiedades normales de la clase. La única excepción es FormsModule, que se importa solo para poder usar ngModel.

7. Estado del componente

Modifica el componente Products para que, en lugar de un producto fijo, tenga:

- Una propiedad con el producto encontrado, que puede no existir (tipo Product | undefined).
- Una propiedad booleana que indique si ya se ha hecho alguna búsqueda.
- Un método privado que reciba un id, busque el producto en PRODUCTS con find() y actualice las dos propiedades anteriores. Las tres formas de búsqueda deben usar este método.

8. Búsqueda con variable de plantilla

- Crea un input de tipo number con una variable de plantilla (por ejemplo #idInput), su label y un botón Buscar.
- Al pulsar el botón o la tecla Enter, pasa el valor del input (idInput.value) a un método del componente que lo convierta a número y haga la búsqueda.

9. Búsqueda con two-way data binding (ngModel)

- A la derecha de la búsqueda anterior, crea otro input con su label y otro botón Buscar.
- Enlaza el input con una propiedad numérica del componente usando [(ngModel)]. Recuerda importar FormsModule en el componente.
- Al pulsar el botón o Enter, llama a un método sin parámetros que busque usando el valor de esa propiedad.

10. Two-way data binding sin ngModel

- A la derecha de la búsqueda con ngModel, crea un tercer input con su label y su botón, que hagan lo mismo pero sin usar ngModel.
- Lleva el valor de una propiedad del componente al input con [value].
- Usa el evento (input) para llamar a un método que reciba el evento ($event) y guarde en la propiedad el valor del input convertido a número (o null si está vacío). No uses el tipo any.

- Al pulsar el botón o Enter, busca usando el valor de esa propiedad.

11. Mostrar el resultado

- Si se encuentra el producto, muestra su tarjeta (imagen, título, categoría, precio y valoración) usando @if.
- Si ya se ha buscado y no existe ningún producto con ese id, muestra el mensaje "No existe ningún artículo con ese id." usando @else if.
- Si todavía no se ha buscado, no se muestra nada.
- Coloca el resultado dentro de un contenedor con aria-live="polite" para que los lectores de pantalla anuncien el cambio.

12. Explicación de cada forma en la página

- Coloca cada forma de búsqueda en su propio recuadro, con un h2 con su nombre, los controles, un párrafo que explique cómo funciona y un bloque de código (<pre><code>) con el fragmento de HTML que usa.
- Los tres recuadros deben verse uno al lado del otro y colocarse uno debajo de otro en pantallas estrechas.
- Añade en el HTML un comentario encima de cada div que explique cómo se hace esa parte.

Criterios de evaluación (segunda parte)

- El proyecto compila sin errores (ng build).
- Las tres formas de búsqueda encuentran el producto correcto, tanto con el botón como con la tecla Enter.
- La búsqueda con variable de plantilla no guarda el valor escrito en ninguna propiedad del componente.
- La segunda búsqueda usa [(ngModel)] con FormsModule.
- La tercera búsqueda no usa ngModel: usa [value] y (input), sin el tipo any.
- La lógica de búsqueda está en un único método que usan las tres formas.
- Se usa @if / @else if para mostrar la tarjeta, el aviso o nada.
- No se usan formularios (<form>, Reactive Forms) ni signals.
- Cada forma tiene visibles en la página su h2, su explicación y su fragmento de código.
- Todos los inputs tienen su label asociado.

Tercera parte: estado y reactividad con zoneless

En esta tercera parte se estudia cómo actualiza Angular la vista cuando cambia el estado de un componente en una aplicación sin zone.js (zoneless), que es lo predeterminado en los proyectos nuevos desde Angular 21. Se compara el estado guardado en propiedades normales de la clase con el estado guardado en signals: primero con un número y después con un array de productos.

En esta parte se introducen los signals (signal(), set() y update()) y la directiva de control de flujo @for. Siguen sin usarse formularios (<form>, Reactive Forms).

13. Componente Reactividad

Dentro de features/reactividad, crea un componente standalone llamado Reactividad, con sus tres archivos separados (.ts, .html y .css) y ChangeDetectionStrategy.OnPush. El componente debe tener:

- Una propiedad numérica normal llamada dato1, con el valor inicial 1.
- Un signal numérico llamado dato2, con el valor inicial 1.
- Para cada dato, un método que lo incremente en una unidad y otro que lo incremente en una unidad dentro de un setTimeout de 2 segundos. El signal se modifica con update(), no asignándole un valor con =.
- Un método vacío, que no cambie ningún dato, para obligar a Angular a revisar el componente al pulsar su botón.
- Cada método que cambie un dato debe escribir su nuevo valor en la consola con console.log().

14. Vista del componente Reactividad

- Un botón «Revisar componente» que llame al método vacío.
- Un div con el título «Sin signal» que muestre el valor de dato1 y dos botones: «Incrementar» e «Incrementar con setTimeout (2 s)».
- Otro div con el título «Con signal» que muestre el valor de dato2 (leyendo el signal como una función) y los mismos dos botones.
- Los dos div deben verse uno al lado del otro, y uno debajo de otro en pantallas estrechas. Cada div debe tener un nombre accesible (role="group" con aria-labelledby) y el valor debe estar en un elemento con aria-live="polite".
- Prueba los botones con la consola del navegador abierta y explica qué ocurre en cada caso: ¿por qué dato1 no se actualiza en la vista al incrementarlo con setTimeout y dato2 sí? ¿Qué pasa al pulsar «Revisar componente»?

15. Array de productos reducido

- En core/mocks, comenta el array PRODUCTS completo y define un nuevo PRODUCTS solo con los 5 primeros productos.
- Comprueba qué ids encuentra ahora el buscador de la segunda parte.

16. Imágenes con NgOptimizedImage en modo fill

- En todos los componentes que muestran productos, sustituye width y height de la imagen por el modo fill de NgOptimizedImage.
- Coloca la imagen dentro de un contenedor con position: relative y un alto fijo, y usa object-fit: contain para que la imagen no se deforme.
- La consola del navegador no debe mostrar el aviso NG02952 sobre la proporción de las imágenes.

17. Componente ProductsArray

Dentro de features/products-array, crea un componente standalone llamado ProductsArray que:

- Guarde en una propiedad normal una copia del array PRODUCTS, de forma que los cambios no modifiquen el array original del mock.

- Muestre todos los productos en tarjetas usando @for, con track por el id del producto y @empty para el caso de que el array esté vacío.
- Coloque las tarjetas en una lista (ul / li) con una rejilla de 5 columnas por fila, que reduzca el número de columnas en pantallas estrechas.
- Tenga un botón «Dar de alta producto» que añada al array con push() un producto definido en una variable del componente (con los datos del producto 10 del array original). El botón se desactiva después del alta, para no repetir el id.
- Tenga un botón «Dar de alta con setTimeout (2 s)» que añada con push() otro producto (con los datos del producto 11) dentro de un setTimeout de 2 segundos, y un botón «Comprobar vista» que no cambie ningún dato.
- Escriba en la consola el array de productos cada vez que se añada un producto.

18. Componente ProductsSignal

- Dentro de features/products-signal, crea un componente standalone llamado ProductsSignal con la misma plantilla y los mismos botones que ProductsArray, salvo «Comprobar vista».
- La propiedad products debe ser un signal de tipo array de Product (signal<Product[]>) con los valores iniciales de PRODUCTS.
- Para dar de alta un producto, usa update() devolviendo un array nuevo con el producto añadido. No uses push() sobre el array del signal.
- Escribe en la consola el array de productos cada vez que se añada un producto.

19. Integración y comparación

- Añade los componentes Reactividad, ProductsArray y ProductsSignal a AppComponent.
- Pulsa «Dar de alta con setTimeout (2 s)» en ProductsArray y en ProductsSignal. Comprueba en cuál aparece la tarjeta sola y en cuál hace falta pulsar «Comprobar vista», y explica por qué.

Criterios de evaluación (tercera parte)

- El proyecto compila sin errores (ng build) y la consola del navegador no muestra avisos de NgOptimizedImage.
- En Reactividad, dato1 es una propiedad normal y dato2 un signal numérico, los dos con el valor inicial 1.
- Al incrementar con un clic, los dos valores se actualizan al momento. Al incrementar con setTimeout, dato2 se actualiza solo y dato1 solo al pulsar «Revisar componente».
- Los signals se modifican con update() o set(), nunca asignándoles un valor con =.
- ProductsArray y ProductsSignal muestran los productos con @for y track por el id.
- ProductsArray añade productos con push() sobre una copia de PRODUCTS. ProductsSignal los añade con update(), devolviendo un array nuevo.
- No se puede dar de alta dos veces el mismo producto.
- Cada incremento y cada alta escriben el estado en la consola.
- Las imágenes de los productos usan NgOptimizedImage en modo fill.
- No se usan formularios (<form>, Reactive Forms).

- El alumno sabe explicar por qué, en una aplicación zoneless, un cambio hecho dentro de un setTimeout solo se refleja en la vista si el estado está en un signal.